

Category:

Model/category:

```
namespace OnlineShop.API.Models
{
    public class Category
    {
        public int Id { get; set; }
        public string Name { get; set; } = string.Empty;
        public List<Product> Products { get; set; } = new();
    }
}
```

Handwritten notes for Category model:
- *شناسه* (ID) next to `Id`
- *اسم* (Name) next to `Name`
- *اسم خاصیت* (Property name) next to `Products`
- *لیستی از نوع برداشت* (List of type Product) next to `Products`
- *به طول پیش فرض خالی* (Default empty length) next to `string.Empty`
- *منظیر کردن* (Similarity) next to `get; set;`
- *گرفتن* (Getting) next to `get;`
- *تنظیم کردن* (Setting) next to `set;`

Categorydto:

```
namespace OnlineShop.API.DTOS.Category
{
    public class CategoryDto
    {
        public int Id { get; set; }
        public string Name { get; set; } = string.Empty;

        public int ProductCount { get; set; }
    }
}
```

Handwritten notes for CategoryDto:
- *اشیاء اشتغال دارا* (Object with occupancy) next to `CategoryDto`
- *تعداد محصولات* (Product count) next to `ProductCount`

Dto/categoryresult:

```
namespace OnlineShop.API.DTOS.Category
{
    public class CategoryResult
    {
        public List<CategoryDto> Categories { get; set; } = new();
        public int TotalCount { get; set; }
        public int TotalPages { get; set; }
    }
}
```

Handwritten notes for CategoryResult:
- *به طول پیش فرض خالی* (Default empty length) next to `new()`
- *اسم خاصیت* (Property name) next to `Categories`
- *لیستی از نوع دسته بندی* (List of type CategoryDto) next to `CategoryDto`
- *تعداد اشیاء* (Number of objects) next to `TotalCount`
- *تعداد اشیاء در صفحات* (Number of objects in pages) next to `TotalPages`
- *سند دسته بندی* (Category type) next to `CategoryResult`

createcategoryDto:

```
namespace OnlineShop.API.DTOS.Category
{
    public class createCategoryDto
    {
        public string Name { get; set; } = string.Empty;
    }
}
```

updatecategoryDto:

```
namespace OnlineShop.API.DTOS.Category
{
    public class updateCategoryDto
    {
        public string Name { get; set; } = string.Empty;
    }
}
```

Interface:

```
using OnlineShop.API.DTOS.Category;
using OnlineShop.API.Models;
namespace OnlineShop.API.Services.Interfaces
{
    public interface ICategoryService
    {
        بسیاری از اینها برمیگرداند
        Task<List<CategoryDto>> GetAllAsync();
        توئی هدر: نام
        Task<CategoryDto?> GetByIdAsync(int id);
        در خروجی می آید
        Task<CategoryDto> CreateAsync(createCategoryDto dto);
        Task UpdateAsync(int id, updateCategoryDto dto);
        Task DeleteAsync(int id);
        باز اتری می رود
        Task<CategoryResult> GetCategoriesAsync(
            string? searchTerm,
            string? sortColumn,
            bool sortAscending,
            int pageNumber,
            int pageSize
        );
    }
}
```

Service:

```
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Logging;
using OnlineShop.API.Data;
using OnlineShop.API.Models;
using OnlineShop.API.Services.Interfaces;
using OnlineShop.API.Helpers;
using OnlineShop.API.DTOS.Category;
```

```
namespace OnlineShop.API.Services;
```

```
public class CategoryService : ICategoryService
```

```
{  
    private readonly IDbContextFactory<AppDbContext> _contextFactory;  
    private readonly ILogger<CategoryService> _logger;
```

```
    public CategoryService(  
        IDbContextFactory<AppDbContext> contextFactory,  
        ILogger<CategoryService> logger)  
    {  
        _contextFactory = contextFactory;  
        _logger = logger;  
    }
```

```
    //Get_list_category-----  
    public async Task<List<CategoryDto>> GetAllAsync()
```

```
{  
    using var context = _contextFactory.CreateDbContext();  
    try  
    {  
        return await context.Categories  
            .Select(c => new CategoryDto  
            {  
                Id = c.Id,  
                Name = c.Name,  
                ProductCount = c.Products.Count()  
            })  
            .ToListAsync();  
    }  
    catch (Exception ex)  
    {  
        logger.LogError(ex, "Error while getting categories");  
        throw;  
    }  
}
```

```
    //GetByIdAsync-----  
    public async Task<CategoryDto?> GetByIdAsync(int id)
```

```
{  
    using var context = _contextFactory.CreateDbContext();  
    try  
    {  
        return await context.Categories  
            .Where(c => c.Id == id)  
            .Select(c => new CategoryDto  
            {  
                Id = c.Id,  
                Name = c.Name,  
                ProductCount = c.Products.Count()  
            })  
            .FirstOrDefaultAsync();  
    }  
    catch (Exception ex)
```

```

{
    _logger.LogError(ex,
        "Error while getting category with id {Id}",
        id);

    throw;
}
}

```

```

//sort_search-pagination-----
public async Task<CategoryResult> GetCategoriesAsync(
    string? categorysearchTerm,
    string? categorysortColumn,
    bool categorysortAscending,
    int categorypageNumber,
    int categorypageSize)

```

و در وی هابی
نه قبول می کنیم

```

using var context = _contextFactory.CreateDbContext();
try
{

```

نمی بیند تا جدول کتگوری

```

    IQueryable<Category> query =
        context.Categories;

```

افتن به جدول کتگوری اما اجرا
نمی کنیم

```

// Search-----
if (!string.IsNullOrEmpty(categorysearchTerm))

```

اگر خالی نباشد

هر کتگوری که شامل سرچ باشد

```

{
    query = query.Where(c =>
        c.Name.Contains(categorysearchTerm));
}

```

```

// Sort-----
query = categorysortColumn switch
{

```

تصمیم گیری

```

    "Name" => categorysortAscending
        ? query.OrderBy(c => c.Name)
        : query.OrderByDescending(c => c.Name),
    "ProductCount" => categorysortAscending
        ? query.OrderBy(c => c.Products.Count())
        : query.OrderByDescending(c => c.Products.Count()),

```

مرتبه بندی صعودی

مرتبه بندی
نزولی

```

    - => categorysortAscending
        ? query.OrderBy(c => c.Id)
        : query.OrderByDescending(c => c.Id)
};

```

به پیش مرتب

```

// Count-----
int totalCount = await query.CountAsync();

```

تعداد کل

تعداد آیتم های کتگوری

```

int totalPages = (int)Math.Ceiling(
    (double)totalCount / categorypageSize);

```

تعداد کل صفحات

جمع اعداد اعشاری را
اوپر بالا گرد می کند


```

    };
}
catch (Exception ex)
{
    _logger.LogError(ex,
        "Error while creating category");

    throw;
}
}
//Edit-category-----
public async Task UpdateAsync(int id, updateCategoryDto dto)
{
    using var context = _contextFactory.CreateDbContext();
    try
    {
        (var category = await context.Categories
            .FirstOrDefaultAsync(c => c.Id == id);

        if (category == null)
        {
            throw new Exception("Category not found.");
        }
        //error-----
        var nameError =
            ValidationHelper.ValidateName(dto.Name);

        if (nameError != null)
        {
            throw new Exception(nameError);
        }
        //----- کد میزنم
        category.Name = dto.Name;
        await context.SaveChangesAsync();
    }
    //ذیرافیسیرا
    catch (Exception ex)
    {
        _logger.LogError(ex,
            "Error while updating category with id {Id}",
            id);

        throw;
    }
}
//Delete_category-----
public async Task DeleteAsync(int id)
{
    using var context = _contextFactory.CreateDbContext();

    try
    {
        var category = await context.Categories
            .Include(c => c.Products)
            .FirstOrDefaultAsync(c => c.Id == id);

        if (category == null)
        {
            throw new KeyNotFoundException(

```

```

        "Category not found.");
    }

    if (category.Products.Any())
    {
        throw new InvalidOperationException(
            "You can't delete a category because it contains products.");
    }

    context.Categories.Remove(category);

    await context.SaveChangesAsync();
}
catch (Exception ex)
{
    _logger.LogError(
        ex,
        "Error while deleting category with id {Id}",
        id);

    throw;
}
}
}

```

Controller:

```

//Usings-----
using Microsoft.AspNetCore.Mvc;
using OnlineShop.API.DTOs.Category;
using OnlineShop.API.Services.Interfaces;

namespace OnlineShop.API.Controllers
{
    دیزاینرهای API
    [ApiController]
    تعیین مسیر برای کنترلر
    [Route("api/[controller]")]
    ارث از کنترلر
    public class CategoriesController : ControllerBase
    {
        نمایندگی
        مجازدن سرویس
        private readonly ICategoryService _categoryService;

        عبارتنامه
        public CategoriesController(
            ICategoryService categoryService)
        {
            _categoryService = categoryService;
        }

        //GET----- لیست می ده
        برای http
        [HttpGet]
        public async Task<ActionResult<List<CategoryDto>>> GetAll()
        {
            var categories = await _categoryService.GetAllAsync();

            return Ok(categories);
        }
    }
}

```

```

//Get_By_Id-----
[HttpGet("{id}")]
public async Task<ActionResult<CategoryDto>> GetById(int id)
{
    var category = await _categoryService.GetByIdAsync(id);

    if (category == null)
    {
        return NotFound();
    }

    return Ok(category);
}

//Get_Search-----
[HttpGet("search")]
public async Task<ActionResult<CategoryResult>> GetCategories(
    string? searchTerm,
    string? sortColumn,
    bool sortAscending = true,
    int pageNumber = 1,
    int pageSize = 10)
{
    var result = await _categoryService.GetCategoriesAsync(
        searchTerm,
        sortColumn,
        sortAscending,
        pageNumber,
        pageSize);

    return Ok(result);
}

//Post-----
[HttpPost]
public async Task<ActionResult<CategoryDto>> Create(
    [FromBody] createCategoryDto dto)
{
    var category =
        await _categoryService.CreateAsync(dto);

    return CreatedAtAction(
        nameof(GetById),
        new { id = category.Id },
        category);
}

//Put-----
[HttpPut("{id}")]
public async Task<IActionResult> Update(
    int id,
    [FromBody] updateCategoryDto dto)
{
    await _categoryService.UpdateAsync(id, dto);

    return NoContent();
}

//Delete-----
[HttpDelete("{id}")]
public async Task<IActionResult> Delete(int id)
{

```

در این بخش، ما یک سری از متدهای API را داریم که برای مدیریت دسته‌ها (Categories) استفاده می‌شوند. در متد `GetById`، ما یک دسته را بر اساس ID دریافت می‌کنیم. در متد `GetCategories`، ما یک لیست از دسته‌ها را دریافت می‌کنیم. در متد `Create`، ما یک دسته جدید ایجاد می‌کنیم. در متد `Update`، ما یک دسته را به‌روزرسانی می‌کنیم. در متد `Delete`، ما یک دسته را حذف می‌کنیم.

در متد `GetCategories`، ما یک سری از پارامترها داریم که می‌توانیم برای فیلتر کردن و مرتب‌سازی نتایج استفاده کنیم. این پارامترها عبارتند از: `searchTerm` (تérیم جستجو)، `sortColumn` (ستون مرتب‌سازی)، `sortAscending` (مرتب‌سازی صعودی)، `pageNumber` (شماره صفحه) و `pageSize` (سایز صفحه).

در متد `Create`، ما یک دسته جدید ایجاد می‌کنیم. ما از متد `CreateAsync` استفاده می‌کنیم تا دسته را در دیتابیس ذخیره کنیم. ما همچنین یک `CreatedAtAction` می‌دهیم تا بتوانیم ID دسته جدید را در پاسخ به کاربر برگردانیم.

در متد `Update`، ما یک دسته را به‌روزرسانی می‌کنیم. ما از متد `UpdateAsync` استفاده می‌کنیم تا دسته را در دیتابیس به‌روزرسانی کنیم. ما همچنین `NoContent()` می‌دهیم تا نشان دهیم که عملیات به‌درستی انجام شده است.

در متد `Delete`، ما یک دسته را حذف می‌کنیم. ما از متد `Delete` استفاده می‌کنیم تا دسته را از دیتابیس حذف کنیم.

```

        await _categoryService.DeleteAsync(id);
        return NoContent();
    }
}

```

f/model/category:

```

export interface Category {
    id: number;
    name: string;
    productCount: number;
}
export interface CategoryResult {
    categories: Category[];
    totalCount: number;
    totalPages: number;
}
export interface CreateCategoryDto {
    name: string;
}

export interface UpdateCategoryDto {
    name: string;
}

```

سمت سرور (C#)	سمت کلاینت (TypeScript)	توضیح
CategoryDto	Category	نمایش دسته بندی
CategoryResult	CategoryResult	نتیجه ی جستجو با صفحه بندی
createCategoryDto	CreateCategoryDto	داده برای ایجاد دسته بندی جدید
updateCategoryDto	UpdateCategoryDto	داده برای به روزرسانی دسته بندی

f/categoryservice:

```

//imports-----
import type {
    Category,
    CategoryResult,
    CreateCategoryDto,
    UpdateCategoryDto
}

```

فراموش
کرد

```

} from "../models/Category/Category";

import { api } from "../api"; فراخوانی ادیس آی ام

//component-----
export const categoryService = {
  //Get_All----- خروجی
  GetAll: async (): Promise<Category[]> => {
    const response = await api.get<Category[]>("/categories");
    return response.data;
  },
  //Get_By_Id----- خروجی
  getId: async (id: number): Promise<Category> => {
    const response = await api.get<Category>(
      `/categories/${id}` در خواست ادیس
    );

    return response.data;
  },
  //Search_sort_pagination-----
  getCategories: async (
    searchTerm?: string,
    sortColumn?: string,
    sortAscending: boolean = true,
    pageNumber: number = 1,
    pageSize: number = 10
  ): Promise<CategoryResult> => {
    const response = await api.get<CategoryResult>(
      "/categories/search",
      {
        params: {
          searchTerm,
          sortColumn,
          sortAscending,
          pageNumber,
          pageSize
        }
      }
    );

    return response.data;
  },
  //Create-----
  create: async (dto: CreateCategoryDto): Promise<Category> => {
    const response = await api.post<Category>(
      "/categories",
      dto
    );

    return response.data;
  },
  //Update-----

```

```

update: async (
  id: number,
  dto: UpdateCategoryDto
): Promise<void> => {
  await api.put(
    `/categories/${id}`,
    dto
  );
},
//Delete-----
delete: async (
  id: number
): Promise<void> => {
  await api.delete(
    `/categories/${id}`
  );
}
};

```

Hook/useCategory:

```

//imports-----
import { useCallback, useEffect, useState } from "react";
import type { Category } from "../models/Category/Category";
import { categoryService } from "../services/categoryService";
//component-----
export const useCategories = () => {
  //state-----
  const [categories, setCategories] = useState<Category[]>([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState("");
  const [search, setSearch] = useState("");
  const [searchTerm, setSearchTerm] = useState("");
  const [currentPage, setCurrentPage] = useState(1);
  const [totalPages, setTotalPages] = useState(1);
  const [sortColumn, setSortColumn] = useState("Id");
  const [sortAscending, setSortAscending] = useState(true);
  //useEffect-----
  useEffect(() => {
    const timer = setTimeout(() => {
      setSearchTerm(search);
      setCurrentPage(1);
    }, 500);
    return () => clearTimeout(timer);
  }, [search]);
  //loadCategory-----
  const loadCategories = useCallback(async () => {
    try {
      setLoading(true);
      setError("");
    }
  });

```

```

const result =
  await categoryService.getCategories(
    searchTerm,
    sortColumn,
    sortAscending,
    currentPage,
    10
  );
// مانی قیمت گذاری
setCategories(result.categories);
setTotalPages(result.totalPages);
}
catch (err) {
  console.error(err);
  setError(
    "Failed to load categories."
  );
}
finally {
  // این نهایت اجرای
  setLoading(false);
}
}, [
  searchTerm,
  currentPage,
  sortColumn,
  sortAscending
]);
// Sort-----
const changeSort = (column:string)=>{
  if(sortColumn === column)
  {
    // اگر دوباره کلیک شده باشد
    setSortAscending(prev=>!prev);
  }
  else
  {
    // true -> false
    // false -> true
    setSortColumn(column);
    setSortAscending(true);
  }

  setCurrentPage(1);
};
//UseEffect-----
useEffect(() => {
  void loadCategories();
}, [loadCategories]);
//Pagination-----
const nextPage = () => {
  setCurrentPage(prev =>
    prev >= totalPages
    // true -> prev
    // false -> prev + 1
  );
};
const previousPage = () => {
  setCurrentPage(prev =>
    prev <= 1

```

```

        ? prev
        : prev - 1
    );
};
//DeleteCategory-----
const deleteCategory = async (
    id: number
) => {
    await categoryService.delete(id);
    await loadCategories();
};
return {
    categories,
    loading,
    error,
    search,
    setSearch,
    currentPage,
    totalPages,
    nextPage,
    previousPage,
    reload: loadCategories,
    deleteCategory,
    changeSort
};
};
};

```

f/component/categorySearch:

داده‌های کامپوننت

```

//imports-----
import "../CategorySearch.css";
//props-----
interface CategorySearchProps {
    searchTerm?: string;
    onSearchChange: (value: string) => void;
}
//component-----
const CategorySearch = ({
    searchTerm,
    onSearchChange,
}: CategorySearchProps) => {
//Body-----
    return (
        <div className="category-search">
            <input
                className="category-search-input"
                type="text"
            />
        </div>
    );
}

```

یک تابع

```

placeholder="Search Categories..."
value={searchTerm}
onChange={ (e) =>
  onChange(e.target.value)
}
  }
  />
</div>

);

};
export default CategorySearch;

```

فقدان رای گیری در این بخش

داده ای

مقدارهای دیگر

f/component/categoryPagination:

```

//imports-----
import "../CategoryPagination.css";

//prop-----
interface CategoryPaginationProps {

  currentPage: number;

  totalPages: number;

  onPrevious: () => void;

  onNext: () => void;

}

//component-----
const CategoryPagination = ({
  currentPage,
  totalPages,
  onPrevious,
  onNext,
}: CategoryPaginationProps) => {

//Body-----
  return (

    <div className="category-pagination">

      <button
        className="pagination-button"
        onClick={onPrevious}
        disabled={currentPage === 1}
      >
        Previous
      </button>

      <span className="pagination-info">

```

```

        Page {currentPage} of {totalPages}
      </span>

      <button
        className="pagination-button"
        onClick={onNext}
        disabled={currentPage === totalPages}
      >
        Next
      </button>

    </div>

  );
};

export default CategoryPagination;

```

f/component/categoryTable:

```

//imports-----
import { useNavigate } from "react-router-dom";
import type { Category } from "../../models/Category/Category";
import "./CategoryTable.css";

//prop-----
interface CategoryTableProps {
  categories: Category[];
  onDelete: (id: number) => void;
  onSort: (column: string) => void;
}

//component-----
const CategoryTable = ({
  categories,
  onDelete,
  onSort,
}: CategoryTableProps) => {
  const navigate = useNavigate();
  if (categories.length === 0) {
    //body-----
    return (
      <div className="category-empty-state">
        <p>
          No categories found.
        </p>

```

```

    </div>

    );
}

return (
    <div className="category-table-container">
        <table className="category-table">

            <thead>

                <tr>

                    <th onClick={() => onSort("Id")}>ID</th>

                    <th onClick={() => onSort("Name")}>Name</th>

                    <th onClick={() => onSort("ProductCount")}>Product</th>

                    <th>Actions</th>

                </tr>

            </thead>

            <tbody>
            {
                categories.map(category => (
                    <tr key={category.id}>
                        برای هر دسته یک رکورد
                        <td>
                            {category.id}
                        </td>

                        <td className="category-name">
                            {category.name}
                        </td>

                        <td>{category.productCount}</td>

                        <td>

                            <div className="category-actions">

```

```

        <button
            className="edit-button"
            onClick={() =>
                navigate(
`/admin/categories/edit/${category.id}`
                )
            }
        >
            Edit
        </button>

        <button
            className="delete-button"
            onClick={() => {
                if (
                    window.confirm(
                        "Are you sure you want to
delete this category?"
                    )
                ) {
                    onDelete(category.id);
                }
            }}
        >
            Delete
        </button>

    </div>

</td>

</tr>

    ))
}

</tbody>

</table>

</div>

);
};

```

```
export default CategoryTable;
```

src/pages/Admin/Categories/Categories.tsx

```
//imports-----
import { useNavigate } from "react-router-dom";
import CategorySearch from
"../../../../components/Admin/Category/CategorySearch";
import CategoryTable from "../../../../components/Admin/Category/CategoryTable";
import CategoryPagination from
"../../../../components/Admin/Category/CategoryPagination";
import { useCategories } from "../../../../hooks/useCategories";
import "./Categories.css";

//component-----
const Categories = () => {

  const navigate = useNavigate();

  const {
    categories,
    loading,
    error,
    search,
    setSearch,
    currentPage,
    totalPages,
    nextPage,
    previousPage,
    deleteCategory,
    changeSort,
  } = useCategories();

  type
  if (loading)
    return <h2 className="loading-text">Loading...</h2>;

  type
  if (error)
    return <h2 className="error-text">{error}</h2>;

  return (

    <div className="categories-page">

      <div className="categories-header">
```

```
<h1 className="categories-title">
  Categories
</h1>

<button
  className="add-category-button"
  onClick={() =>
    navigate("/admin/categories/create")
  }
>
  + Add Category
</button>
```

```
</div>
```

```
<CategorySearch
  searchTerm={search}
  onSearchChange={setSearch}
/>
```

Handwritten notes: *prop = value* (with an arrow pointing from *prop* to *searchTerm={search}*) and *prop* (with an arrow pointing from *prop* to *onSearchChange={setSearch}*).

```
<CategoryTable
  categories={categories}
  onDelete={deleteCategory}
  onSort={changeSort}
/>
```

Handwritten notes: *prop = value* (with an arrow pointing from *prop* to *categories={categories}*), *prop* (with an arrow pointing from *prop* to *onDelete={deleteCategory}*), and *prop* (with an arrow pointing from *prop* to *onSort={changeSort}*).

```
<CategoryPagination
  currentPage={currentPage}
  totalPages={totalPages}
  onPrevious={previousPage}
  onNext={nextPage}
/>
```

Handwritten notes: *prop = value* (with an arrow pointing from *prop* to *currentPage={currentPage}*), *prop* (with an arrow pointing from *prop* to *totalPages={totalPages}*), *prop* (with an arrow pointing from *prop* to *onPrevious={previousPage}*), and *prop* (with an arrow pointing from *prop* to *onNext={nextPage}*).

```

        />

    </div>

    );
};

export default Categories;

```

App.tsx:

```

//Imports-----
import { Routes, Route } from "react-router-dom";
import AdminLayout from "../layout/AdminLayout/AdminLayout";
//product-----
import Products from "../pages/Admin/Products/Products";
import CreateProduct from "../pages/Admin/Products/CreateProduct";
import EditProduct from "../pages/Admin/Products/EditProduct";
import ProductDetail from "../pages/Admin/Products/ProductDetail";
//category-----
import Categories from "../pages/Admin/Categories/Categories";
import CreateCategory from "../pages/Admin/Categories/CreateCategory";
import EditCategory from "../pages/Admin/Categories/EditCategory";

function App() {
    return (
        <Routes>
            <Route path="/admin" element={<AdminLayout />}>
                {/* Products----- */}
                <Route path="products" element={<Products />} />
                <Route path="products/create" element={<CreateProduct />} />
                <Route path="products/edit/:id" element={<EditProduct />} />
                <Route path="products/:id" element={<ProductDetail />} />
                {/* Category----- */}
                <Route path="categories" element={<Categories />} />
                <Route path="categories/create" element={<CreateCategory
/>}/>

                <Route path="categories/edit/:id" element={<EditCategory
/>}/>

            </Route>
        </Routes>
    );
}

export default App;

```

CreateCategory:

```

//imports-----
import { useState } from "react";
import { useNavigate } from "react-router-dom";
import { categoryService } from "../../services/categoryService";
import "../CreateCategory.css";
//component-----
const CreateCategory = () => {
  //state-----
  const navigate = useNavigate();
  const [name, setName] = useState("");
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState("");

  //handle_submit----- مدیریت ارسال فرم
  const handleSubmit = async (
    e: React.FormEvent<HTMLFormElement>
  ) => {
    e.preventDefault();
    setLoading(true);
    setError("");

    try {
      await categoryService.create({
        name
      });

      navigate("/admin/categories");
    } catch (err) {
      console.error(err);
      setError(
        "Failed to create category."
      );
    }
    finally{
      setLoading(false);
    }
  };

  //Body-----
  return (
    <div className="category-page">
      <div className="category-header">
        <h1 className="category-title">

```

یکنی
 ارسال فرم
 ارسال

مدیریت ارسال فرم
 بعد از نام و نام خانوادگی
 پس از اتمام موفق فرم را

```

        Add Category
      </h1>

      <p className="category-subtitle">
        Create a new product category
      </p>

      <button
        className="category-back-button"
        type="button"
        onClick={() => navigate("/admin/categories")}
      >
        ← Back
      </button>

    </div>

```

```

    <form
      className="category-form"
      onSubmit={handleSubmit}
    >
      اگر آلوده داشته باشد
      {error &&
        <div className="form-error">
          {error}
        </div>
      }

      <div className="form-group">

        <label>
          Category Name
        </label>

        <input
          type="text"
          value={name}
          onChange={
            هر بار که کاربر چیزی تایپ کند
            e => setName(e.target.value)
            به روز رسانی می شود
          }
          required
          اجباری است
        >

      </div>

      <div className="form-actions">

        <button
          className="category-cancel-button"
          type="button"

```

```

        onClick={() =>
            navigate("/admin/categories")
        }
    >
        Cancel
    </button>

    <button
        className="category-submit-button"
        type="submit"
        disabled={loading}
    >
        {
            loading
            true ? "Creating..."
            false : "Create Category"
        }
    </button>

</div>

</form>
</div>

);
};

export default CreateCategory;

```

EditCategory:

```

//imports-----
import {
    useEffect,
    useState
} from "react";

import {
    useNavigate,
    useParams
} from "react-router-dom";

import { categoryService } from "../../services/categoryService";
import "../CreateCategory.css";

//component-----
const EditCategory = () => {
    //state-----
    const { id } = useParams<{id:string}>();

```

→ /admin/categories/edit/5

```

const navigate = useNavigate();
const [name, setName] = useState("");
const [loading, setLoading] = useState(true);
const [saving, setSaving] = useState(false);
const [error, setError] = useState("");

```

```

//UseEffect-----
useEffect(()=>{

```

```

    const load = async()=>{
        if(!id)
            return;

```

id وجود ندارد

```

        try{
            const category =
                await categoryService.getById(
                    Number(id)
                );
            setName(category.name);
        }
        catch{
            setError(
                "Failed to load category."
            );
        }
        finally{
            setLoading(false);
        }
    };

```

موفق

نام را می‌گیرد

```

    void load();
    }, [id]);

```

الای تغییر

داده‌ها را بگیر

```

//handle_submit-----
const handleSubmit = async(
    e:React.FormEvent<HTMLFormElement>
)>=>{
    e.preventDefault();

```

```

        if(!id)
            return;

        setSaving(true);
        setError("");

        try{

            await categoryService.update(
                Number(id),
                {
                    name
                }
            );

            navigate("/admin/categories");

        }
        catch{

            setError(
                "Failed to update category."
            );

        }
        finally{

            setSaving(false);

        }

    };

    if(loading)
    {
        return <h2>Loading...</h2>;
    }

    //Body-----
    return (

        <div className="category-page">

            <div className="category-header">

                <h1 className="category-title">
                    Edit Category
                </h1>

                <p className="category-subtitle">
                    Update category information

```

```
</p>

<button
  className="category-back-button"
  type="button"
  onClick={ () =>
    navigate("/admin/categories")
  }
>
  ← Back
</button>

</div>

<form
  className="category-form"
  onSubmit={handleSubmit}
>

  {
    error &&
    <div className="form-error">
      {error}
    </div>
  }

  <div className="form-group">

    <label>
      Category Name
    </label>

    <input
      type="text"
      value={name}
      onChange={
        e=>setName(e.target.value)
      }
      required
    />

  </div>
```

```

        <div className="form-actions">

            <button
                className="category-cancel-button"
                type="button"

                onClick={ () =>
                    navigate("/admin/categories")
                }

            >
                Cancel
            </button>

            <button
                className="category-submit-button"

                type="submit"

                disabled={saving}

            >

                {
                    saving
                    ? "Saving..."
                    : "Save Changes"
                }

            </button>

        </div>

    </form>

</div>

);
};

export default EditCategory;

```

CategorySelect for Product(edit/create):

```

import { useEffect, useState } from "react";
import { categoryService } from "../../services/categoryService";
import type { Category } from "../../models/Category/Category";
import "./CategorySelect.css"
interface CategorySelectProps {
  value: number;
  onChange: (id: number) => void;
}

const CategorySelect = ({
  value,
  onChange,
}: CategorySelectProps) => {

  const [categories, setCategories] = useState<Category[]>([]);
  const [search, setSearch] = useState("");
  const [open, setOpen] = useState(false);
  const [loading, setLoading] = useState(false);

  useEffect(() => {

    const timer = setTimeout(async () => {

      try {

        setLoading(true);

        const result =
          await categoryService.getCategories(
            search,
            "Name",
            true,
            1,
            50
          );

        setCategories(result.categories);

      }
      catch(err) {

        console.error(err);

      }
      finally {

        setLoading(false);

      }

    }, 300);

    return () => clearTimeout(timer);
  }

```

افيت باءه
لنوي ليه

```
}, [search]);
```

```
const selectedCategory =  
  categories.find(c => c.id === value);
```

طه زدن روی

```
return (
```

```
  <div className="category-select">
```

```
    <input
```

```
      type="text"
```

```
      placeholder="Search category..."
```

```
      value={
```

آندۀ اثر
آندۀ وجود داشت
آندۀ با آندۀ بد
آندۀ متن جستجو و نشون به

```
        open
```

```
        ? search
```

```
        : selectedCategory?.name ?? ""
```

آندۀ روئ کرج لیل کت

```
      } onFocus={() => setOpen(true)}  
      onChange={e => {
```

```
        setSearch(e.target.value);  
        setOpen(true);
```

```
      }}  
    />
```

```
    {
```

```
      open && (
```

```
        <div className="category-dropdown">
```

```
          {
```

```
            loading &&
```

```
            <p>
```

```
              Loading...
```

```
            </p>
```

```
          }
```

```
          {
```

```
            categories.map(category => (
```

```
              <div
```

```
                key={category.id}
```

```
                className="category-option"
```

```
                onClick={() => {
```

```
                  onChange(category.id);
```

```
                  setOpen(false);
```

```
                  setSearch("");
```

```
                >
                    {category.name}
                </div>
            ))
        }

    </div>

    )
}

</div>

);

export default CategorySelect;
```
